FeBEx: A P4-Programmable Federated Backhaul Exchange

CS6204 Project

Rajashekar Reddy Chinthalapani & Savitha **Viswanadh** Kandala

Why do we need federated IoT networks?

Traditional IoT networks are centralized. One operator deploys all gateways, runs the servers, and controls device access. This model is expensive to scale and leaves large areas uncovered

1. High Deployment Costs

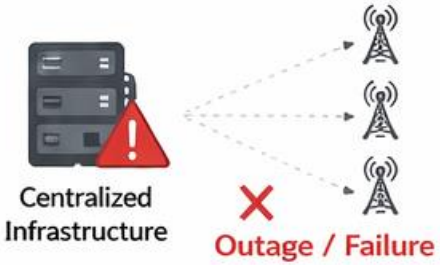
Building and maintaining nationwide or city-wide IoT coverage requires significant capital and operational expenditure.

2. Coverage Gaps

No single operator can deploy everywhere. Rural, remote, or low-density areas often remain uncovered.

3. Single Point of Failure

A centralized network operated by a single entity is vulnerable to outages, attacks, or business disruptions.



4. Vendor Lock-In

Proprietary technologies and closed ecosystems limit interoperability, increase switching costs, and reduce

What if anyone could deploy a gateway and contribute to a shared, open IoT network?

Federated IoT Networks: Helium network

Federated IoT Networks: A Distributed, Collaborative Alternative

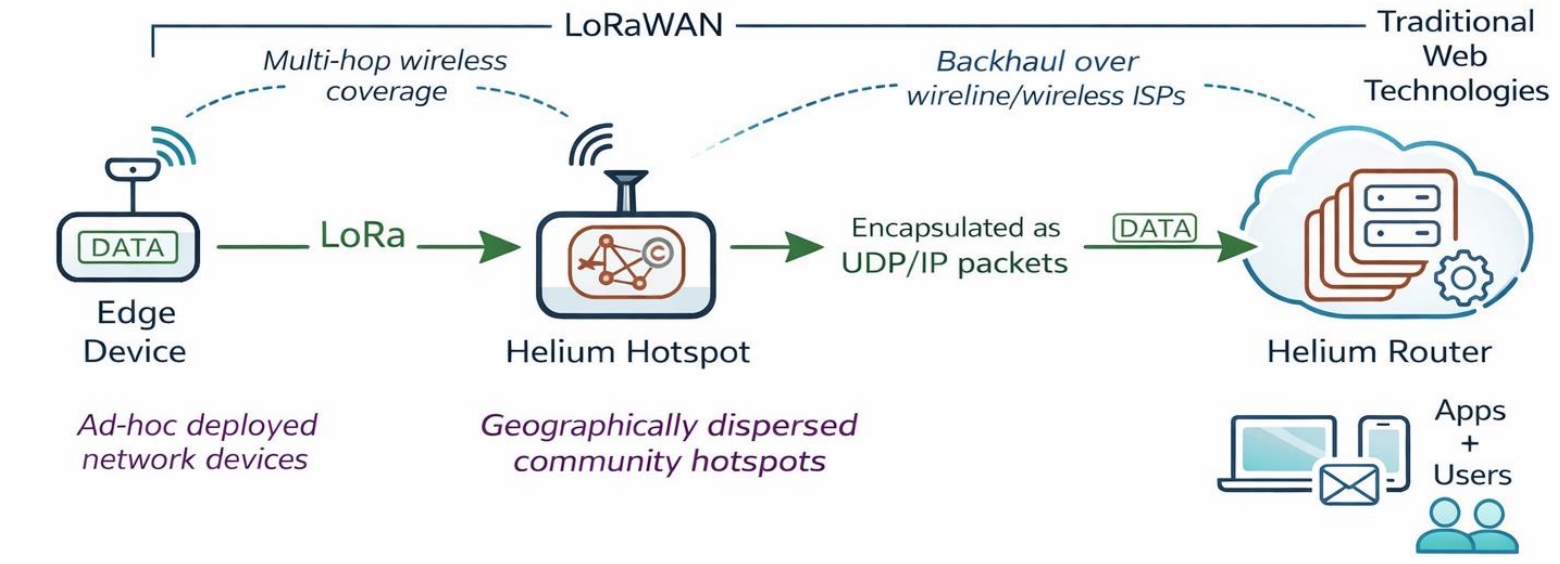
Shared Deployment Costs
- Operators share costs
- Lower CapEx/OpEx

Enhanced Coverage
- Gaps plugged by
- Strong camera nates

No Single Point of Failure
- More redundancy & resillence
- Standards, APIs, multi-vendor

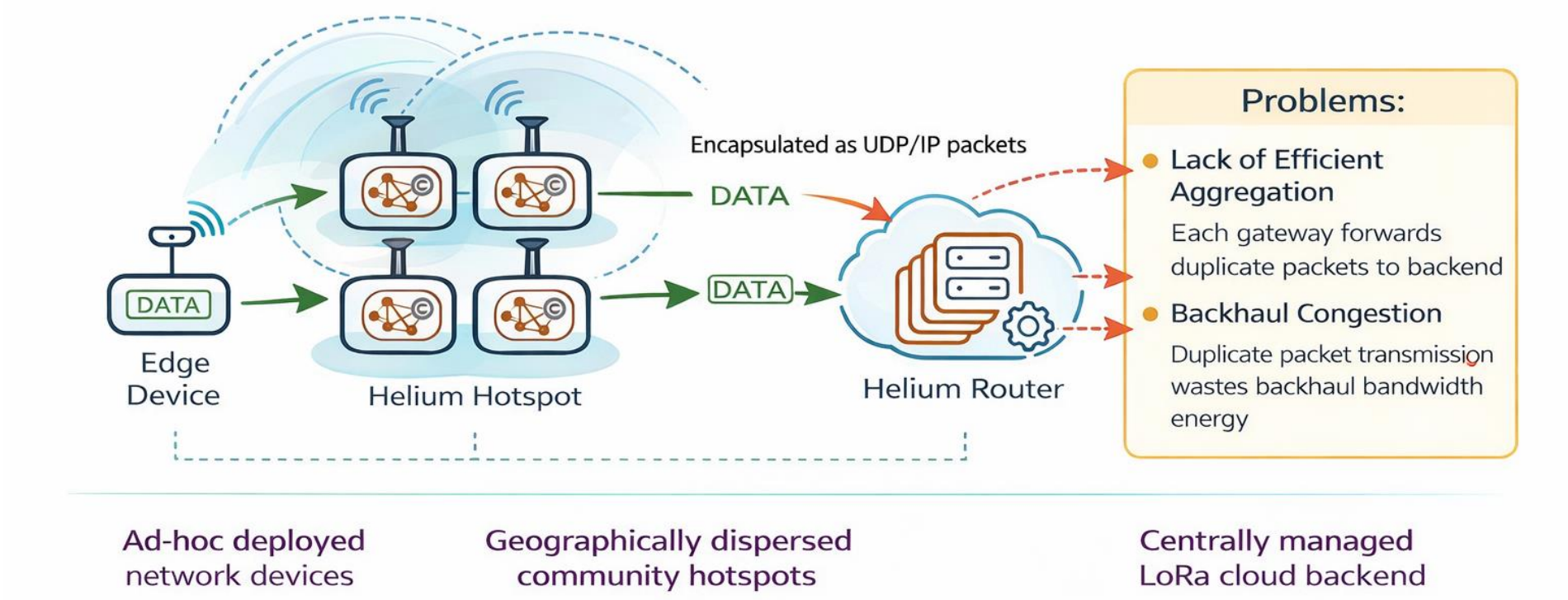
Communicy
- Decentralized
- Catalocative overshots

Federation enables scalable, resillent, cost-effective IoT at city and global scale.



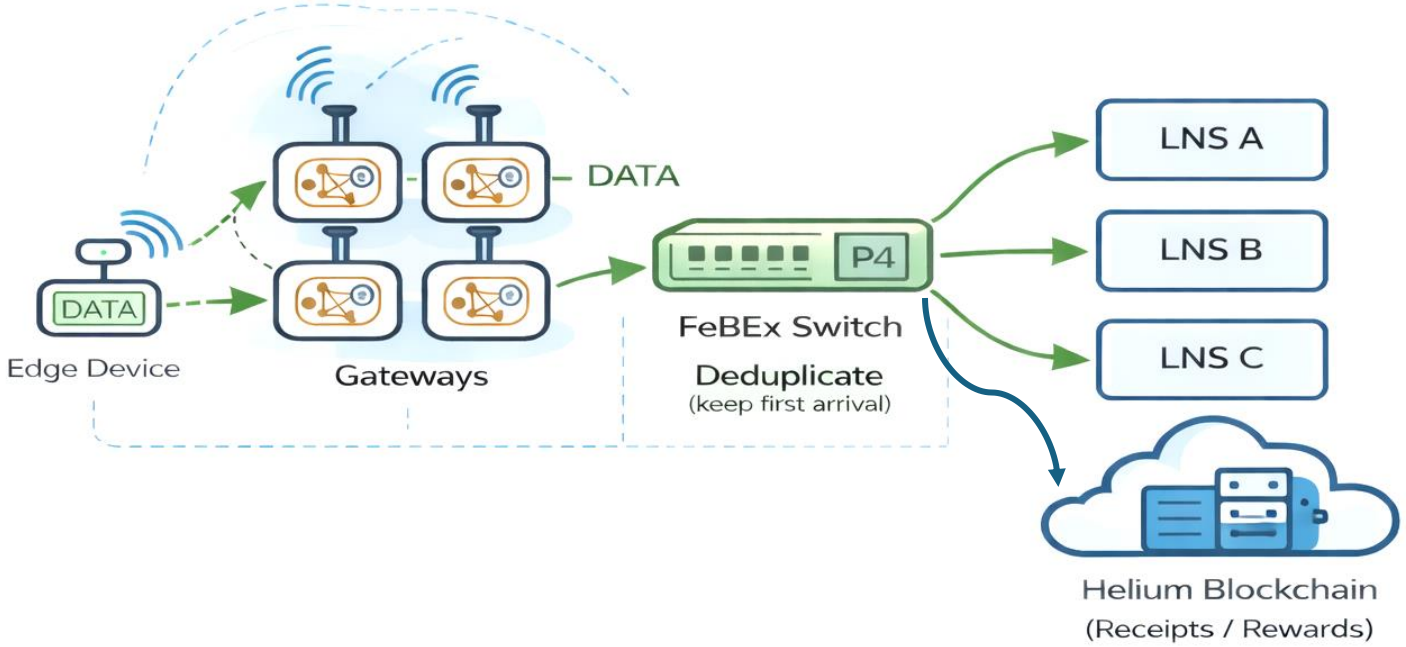
The Federated LoRaWAN Challenge

But federated networks bring the redundancy challenge. Overlapping hotspot coverage creates massive packet duplication and stresses the backhaul

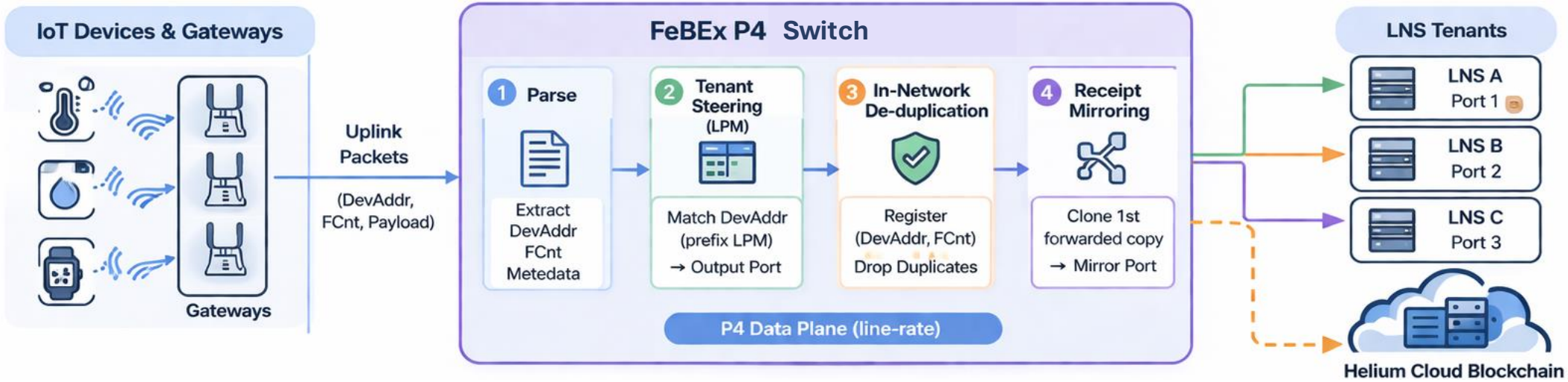


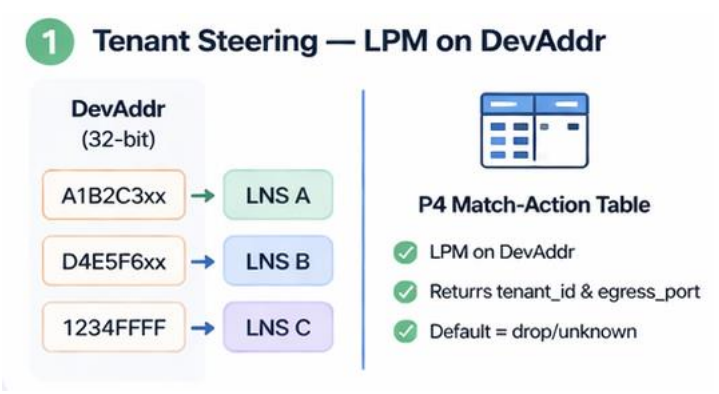
FeBEx for in-network deduplication of LoRaWAN uplinks

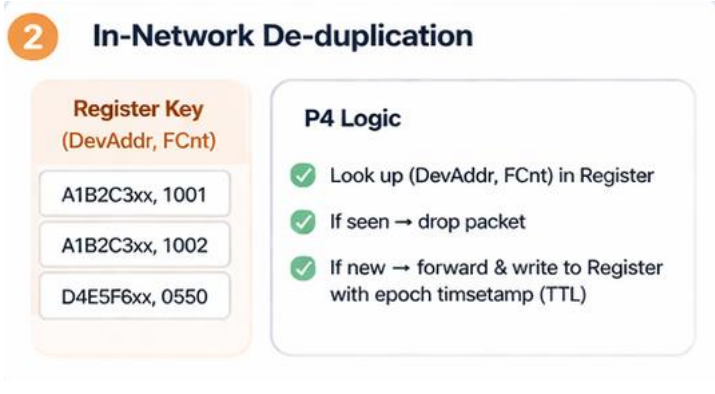
We draw inspiration from IXP architecture. FeBEx moves the deduplication and tenant steering from the cloud to the network at an aggregation point

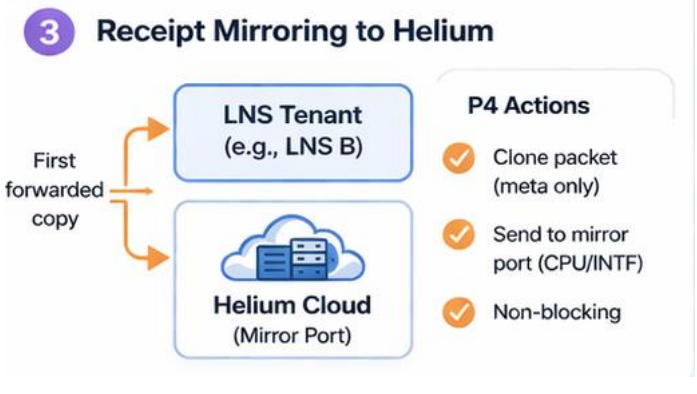


Architecture overview and implementation









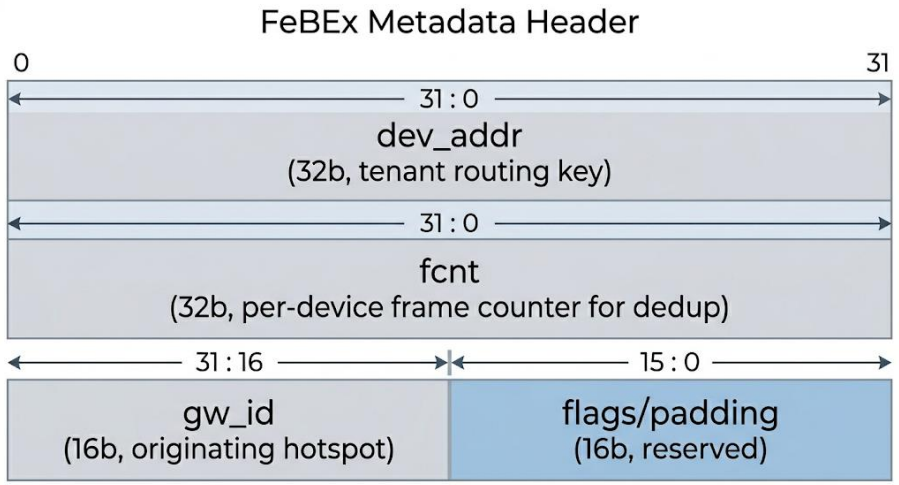
Packet structure

Not feasible for in-switch decryption

- Custom 12-byte FeBEx metadata header appended to UDP payload
- Four fields:
  - dev_addr (32b, tenant routing key),
  - fcnt (32b, per-device frame counter for dedup),
  - gw_id (16b, originating hotspot),
  - flags/padding (16b, reserved)
- Layered as:

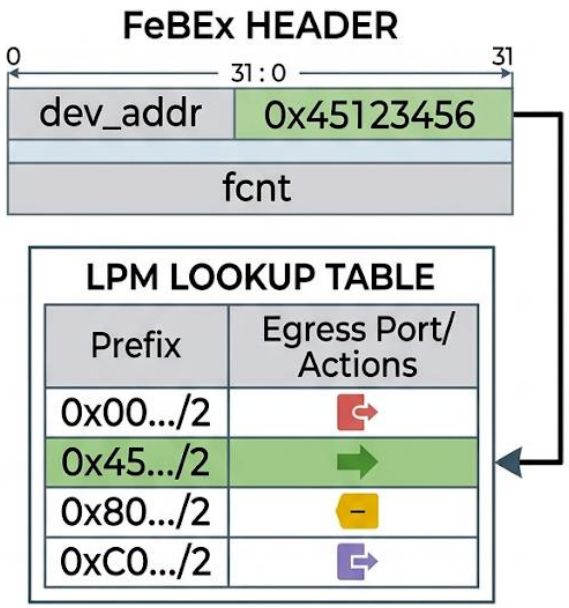
  Ethernet | IPv4 | UDP | FeBExMeta | payload
- Built with scapy custom layer class; parsed in-switch by the P4 parser chain

LoRaWAN PHY payload is base64-encoded



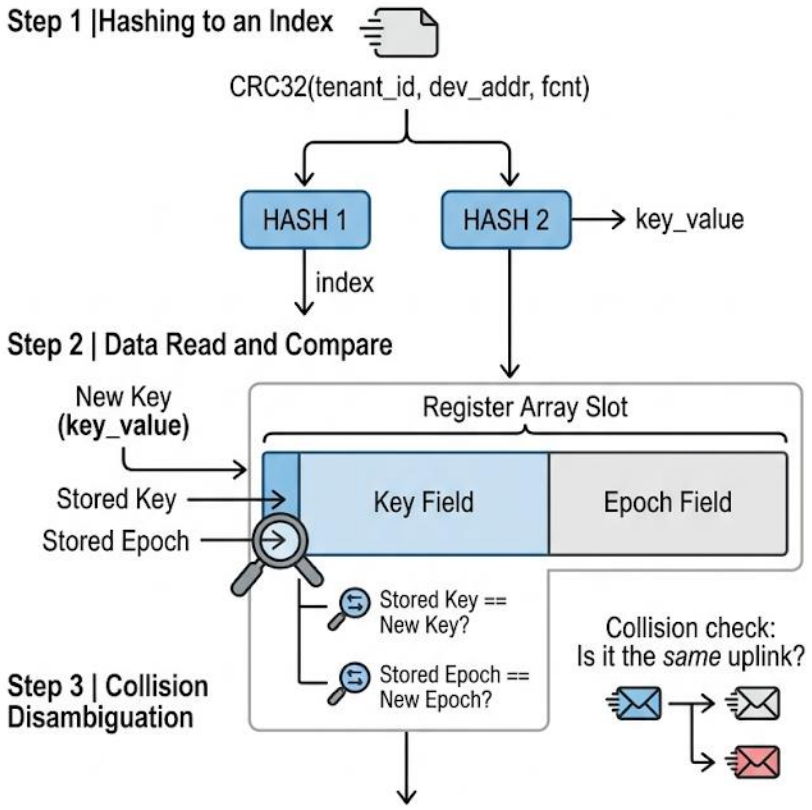
Tenant steering (LPM table)

- Match key: dev_addr with LPM -- space partitioned across M tenants using $\log_2(M)$ prefixes
- Action: set_tenant sets egress port, rewrites Ethernet dst MAC (P4 switch performs no ARP), stores tenant_id in pipeline metadata
- Default action: drop — unrecognised prefixes are discarded
- Controller pre-installs all prefix entries at startup; no per-packet control-plane involvement



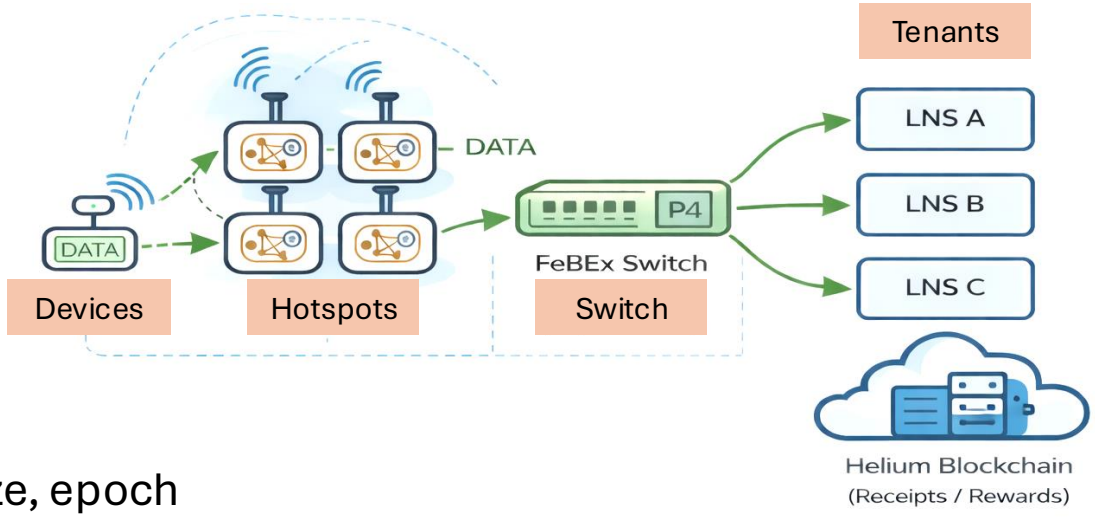
Register-based deduplication

- Two parallel register arrays indexed by 3-tuple: to store verification hash and epoch
- A packet is a duplicate iff
  - stored_epoch == current_epoch AND
  - stored_key == key_value
- Avoids false positives from hash index collisions
- Controller increments a 16-bit counter every N seconds; stale-epoch entries are overwritten

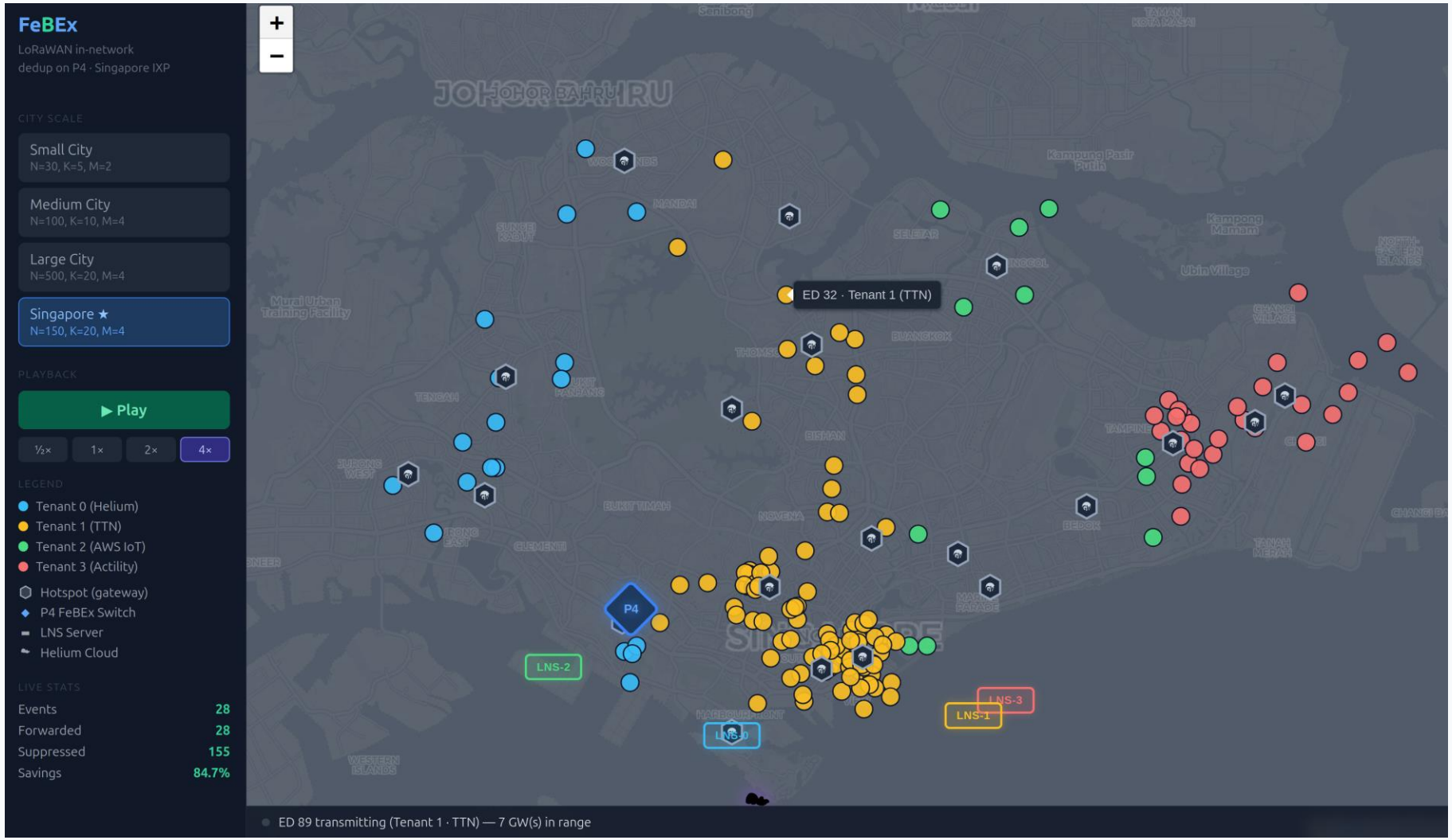


Evaluation methodology

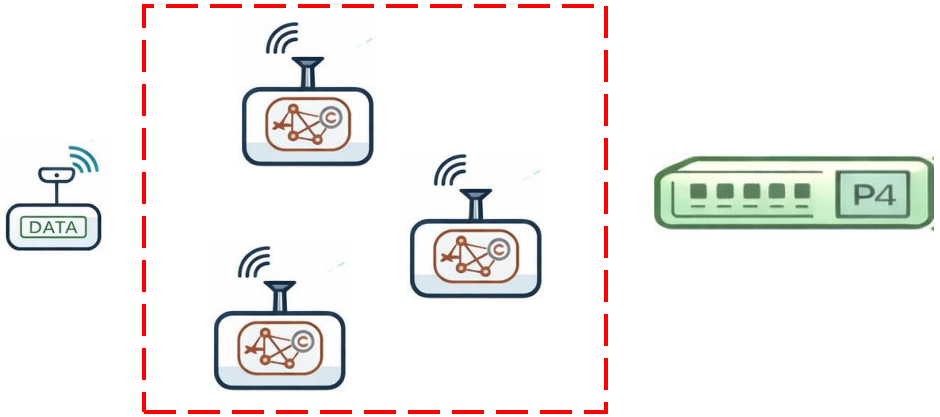
- We simulate different locality scales (poisson):
  - Small: N=5 devices, K=3 hotspots, M=2 tenants
  - Medium: N=100 devices, K=10 hotspots, M=4 tenants
  - Large: N=500 devices, K=50 hotspots, M=4 tenants
- Baseline (dedup OFF) vs. FeBEx (dedup ON)
- Metrics logged at LNS receivers and cloud receiver
- Experiments sweep over duplicate factor, register size, epoch interval, and network scale



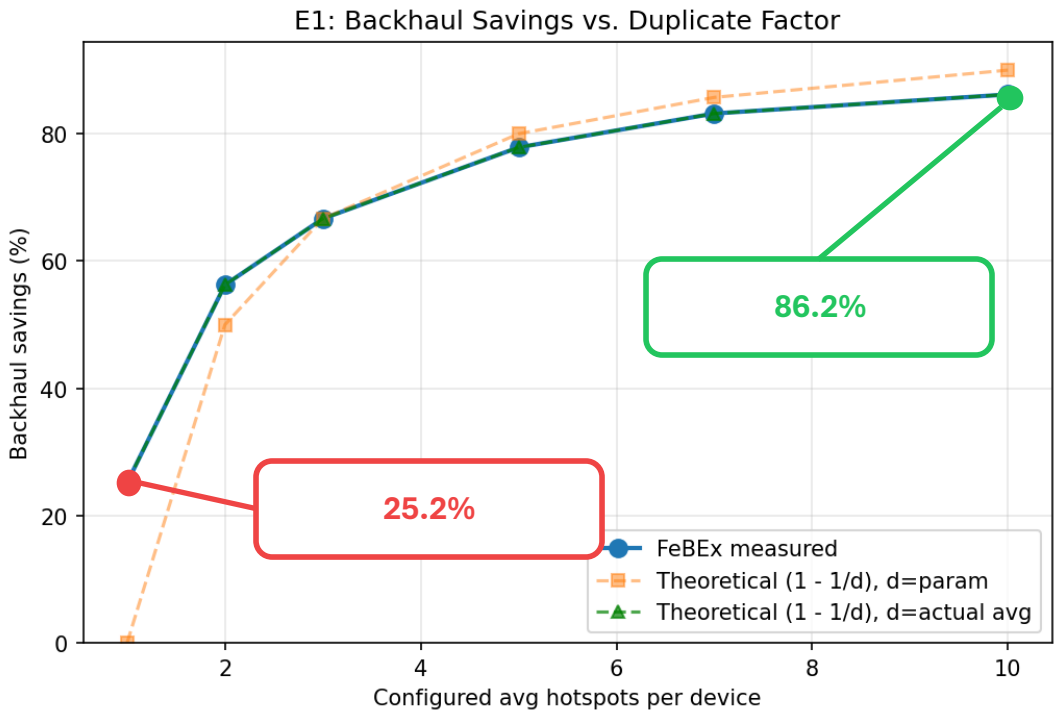
Live Visualization Demo



1: Backhaul savings vs Duplicate Factor



- Metric: savings % = $1 - \frac{packets_{unique}}{packets_{total}}$
- FeBEx savings increase with overlap: from ~25% at low duplication to **~86%** at high duplication
- Measured savings stay close to theoretical expectations
- **Takeaway:** higher gateway overlap directly translates to major backhaul cost reduction



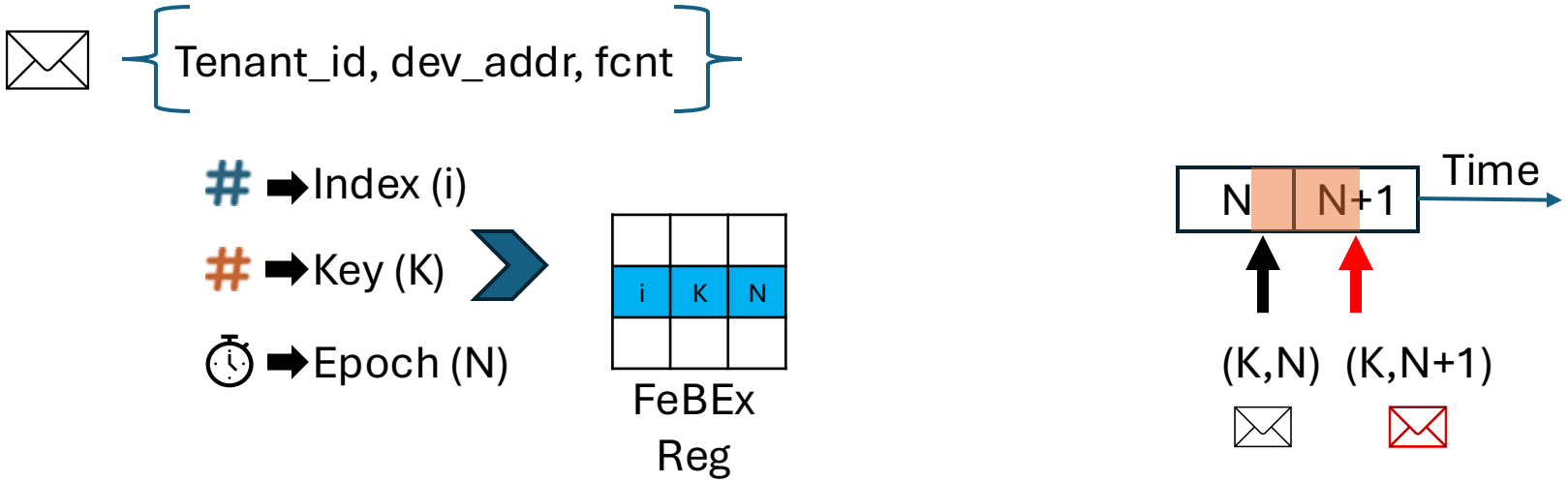
Params: N=100, K=10, M=2 | uplinks/device=50 | avg_dup sweep 1→10

Boundary Leakages

The FeBEx register stores (key, epoch) per slot. A packet is a **duplicate** if stored_key == key AND stored_epoch == current_epoch. The race occurs when:

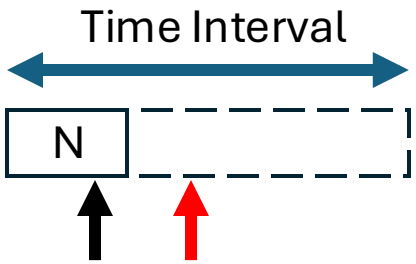
- Copy 1 arrives at epoch N → register written: (key=K, epoch=N) → Forwarded
- Controller rotates epoch to N+1 (after epoch_interval seconds)
- Copy 2 arrives at epoch N+1 → reads stored_epoch=N, sees N ≠ N+1 → slot treated as stale → forwarded as fresh (boundary leakage)

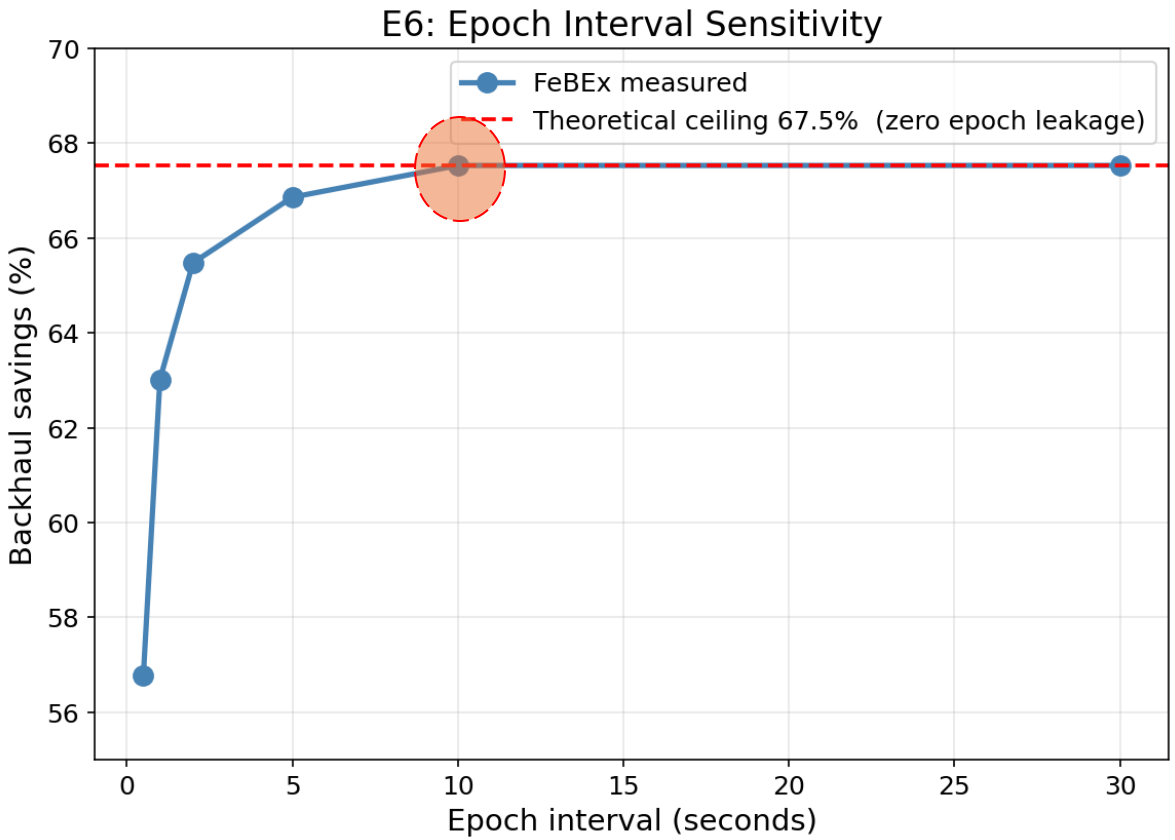
Upto 4.2% duplicate packets transmitted during stress tests



2: Epoch Interval Sensitivity

- Very short epochs reduce suppression quality (more boundary leakage)
- Longer intervals converge to near-maximum observed savings
- **Takeaway**: choose a balanced epoch (e.g., mid-range) to avoid boundary artifacts while keeping freshness/ low false-positives

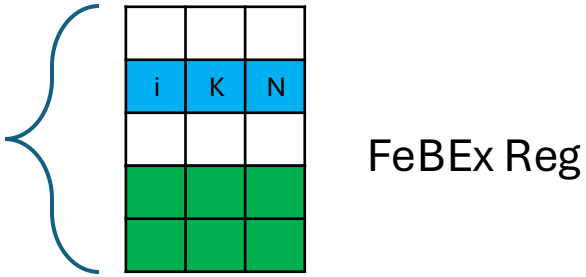




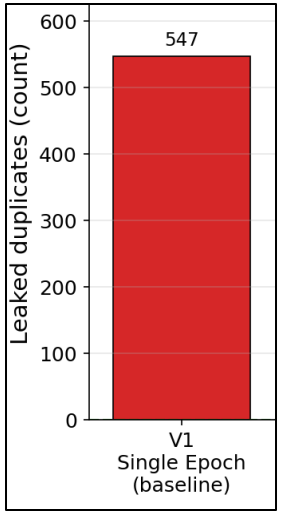
Params: N=100, K=10, M=2 | epoch sweep 0.5,1,2,5,10,30 s

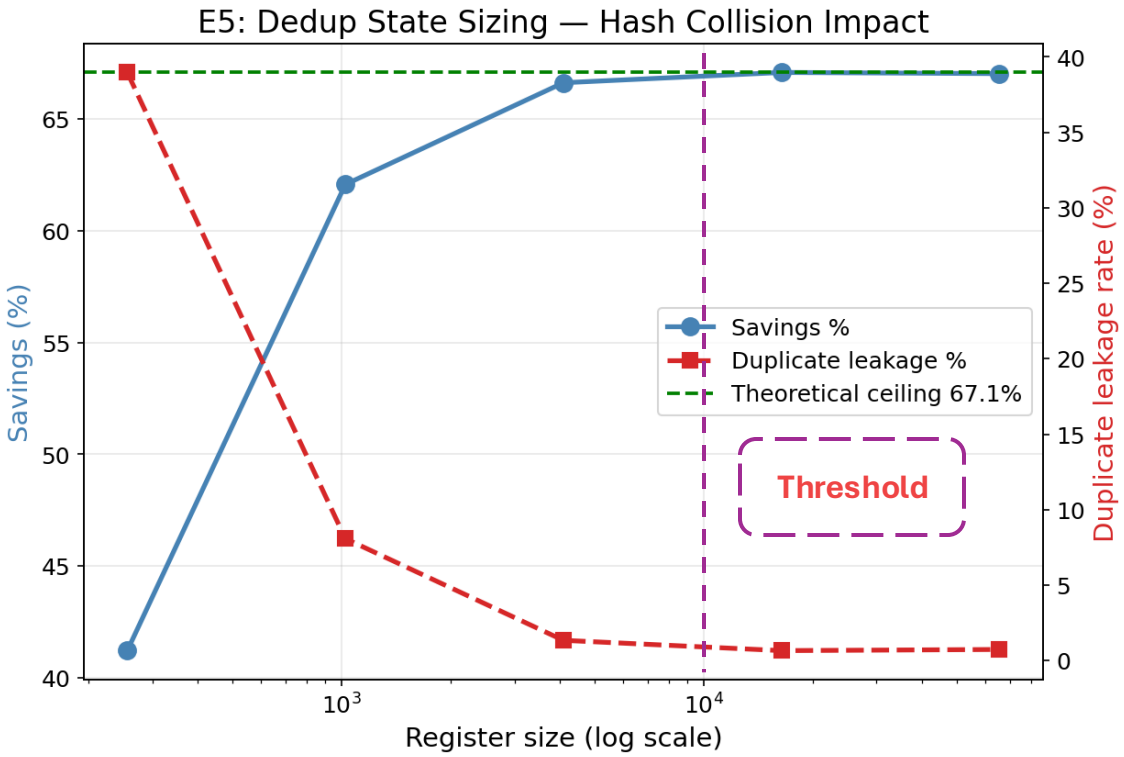
3: State Sizing / Register Collisions

- Small dedup state causes significant duplicate leakage and lower savings
- Increasing register size sharply reduces leakage, then reaches a plateau
- **Takeaway**: right-sizing dedup state is critical; under-provisioning hurts effectiveness



Leakages still exist!

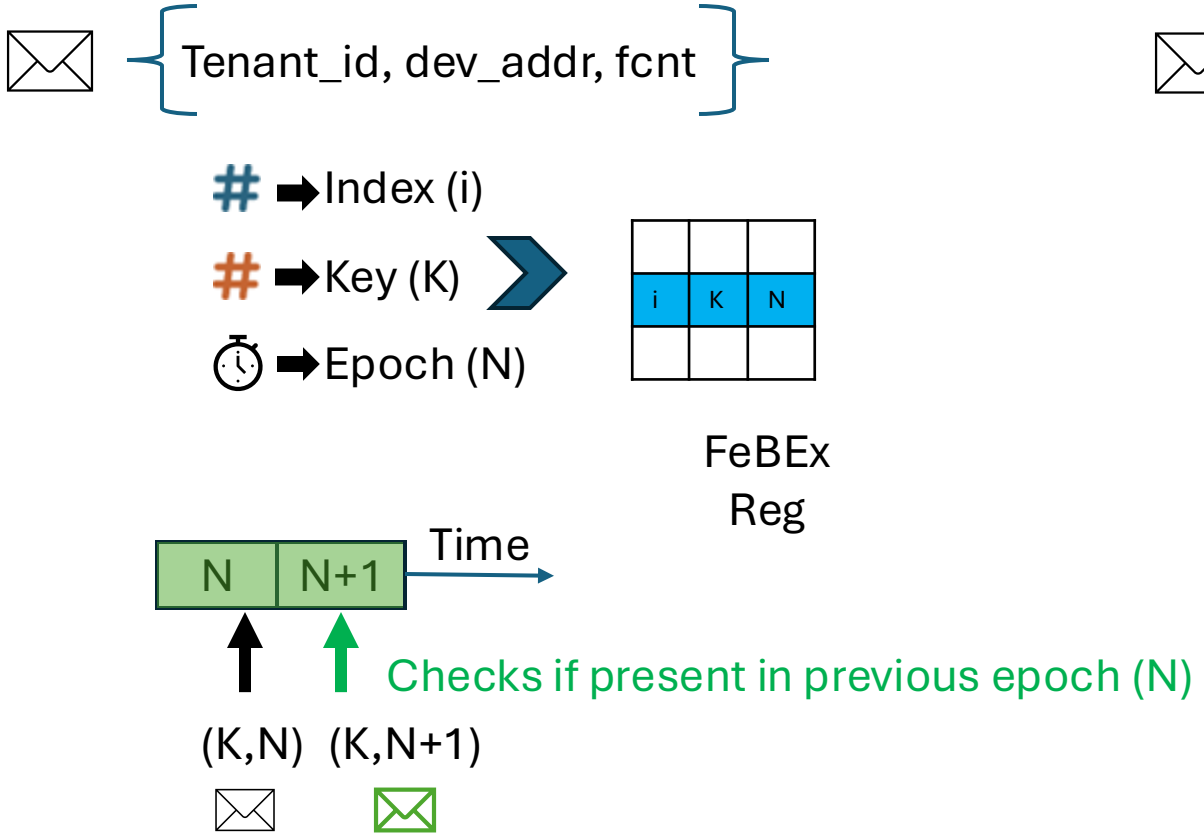




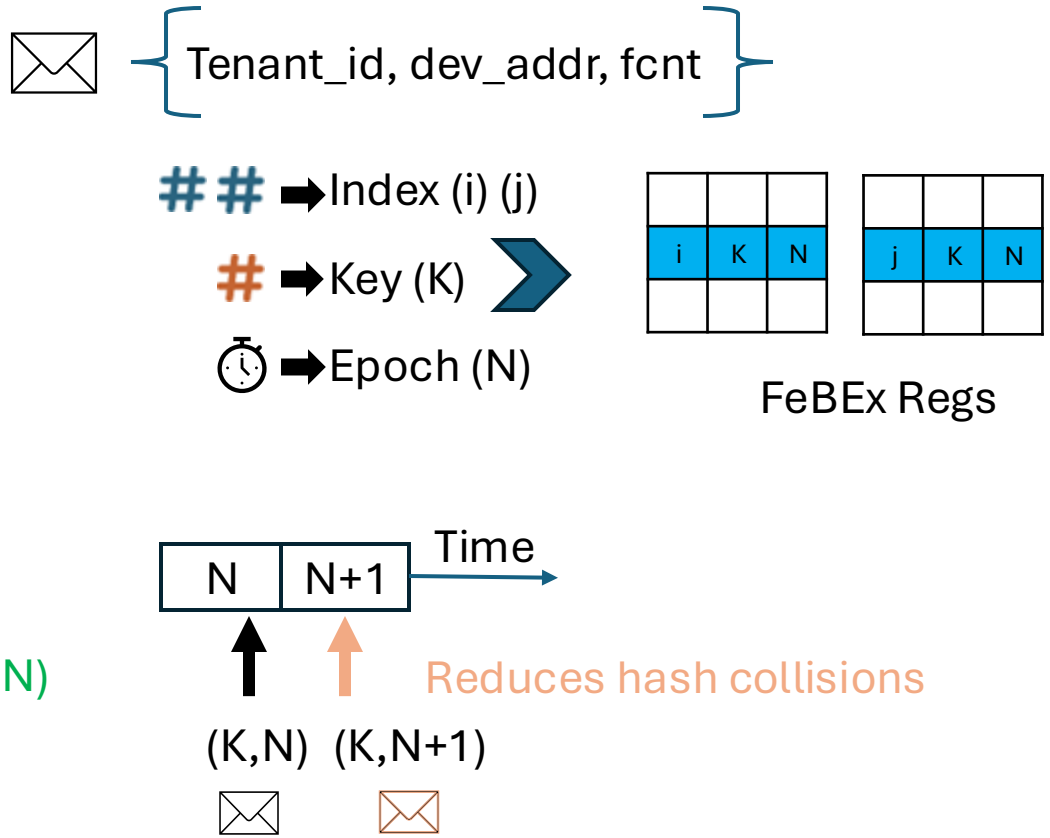
Params: N=100, K=10, M=2 | register sweep 256→65,536

Boundary Leakages

V2: Sliding Window



V3: Dual-register Bloom

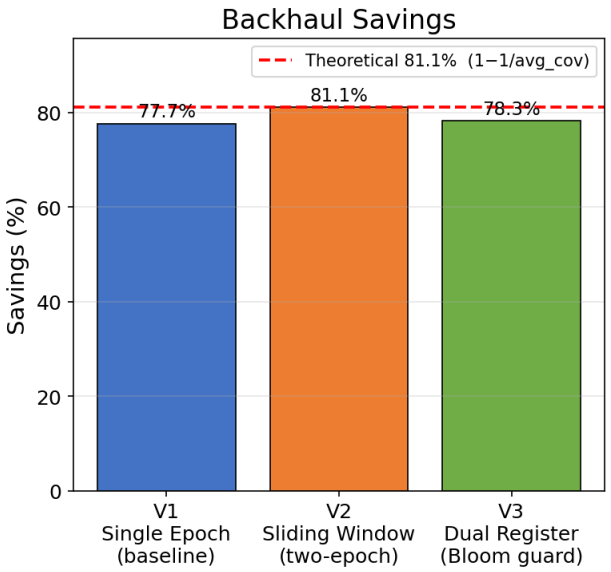


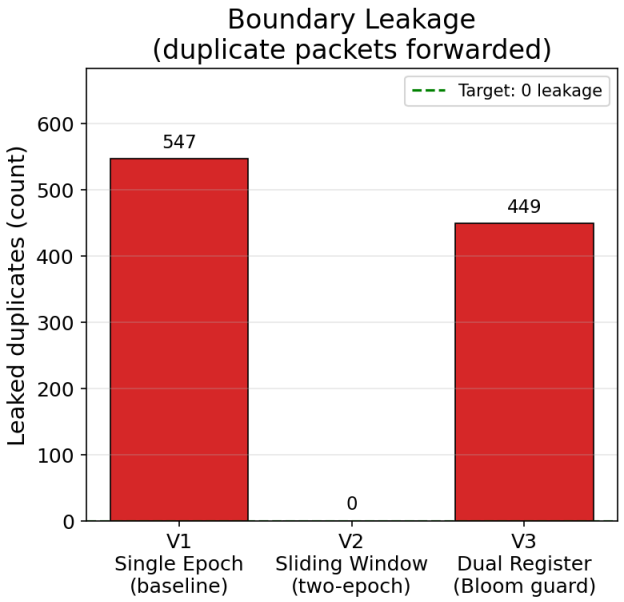
Boundary Leakages

	V1	V2	V3
Boundary leakage	Present	**Eliminated**	Same as V1
Hash collision false-positives	O(1/N)	O(1/N)	**$O(1/N^2)$**
Register memory	**1×**	1× + 1 extra element	2×
Pipeline reads/writes	**2 reads, 2 writes**	3 reads, 2 writes	4 reads, 4 writes
Controller change	**None**	prev_epoch write before rotation	**None**

4: Variant Comparison under Boundary Stress

- V2 (sliding-window) achieves best performance, reaching theoretical ceiling behavior
- V1 and V3 remain lower due to residual boundary leakage
- **Takeaway**: sliding-window epoch eliminates boundary leakage. V2+V3 best for real-world

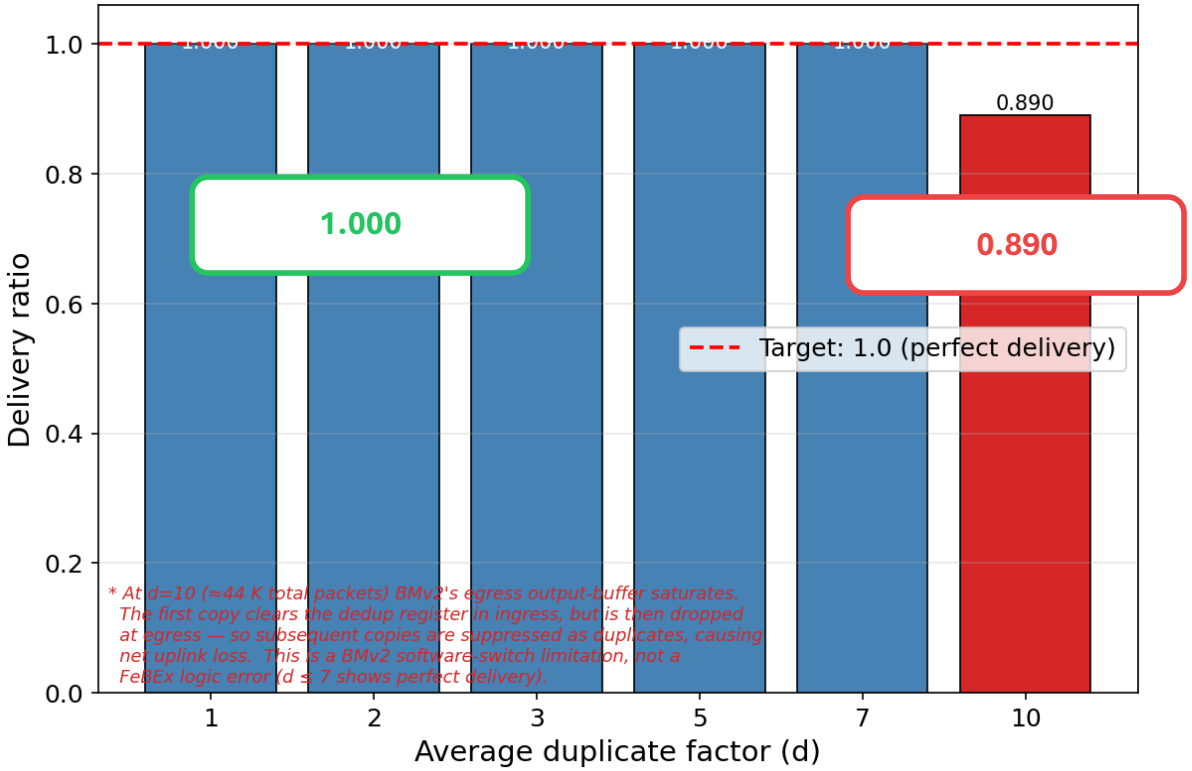




Stress params: N=100, K=10, M=2 | epoch=1s | inter-arrival=500ms | avg_cov=5.3

5: Correctness -- Unique Uplink Delivery

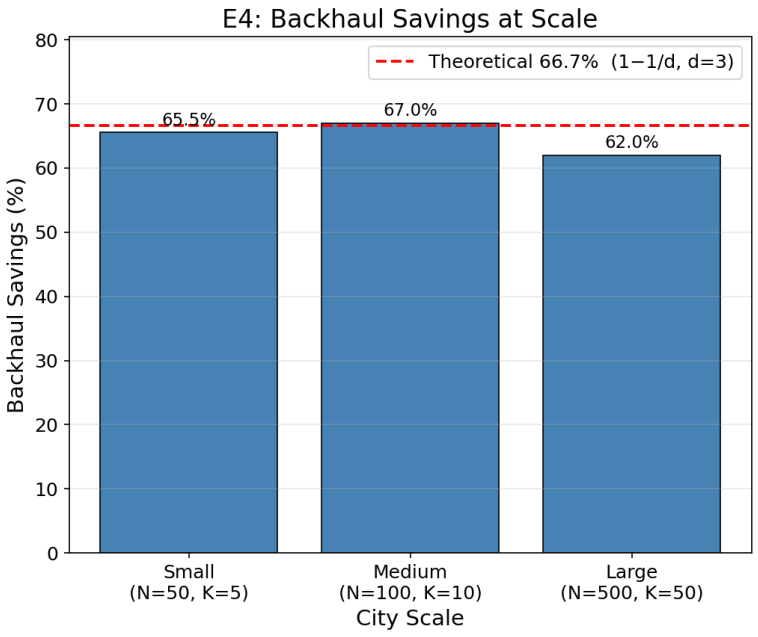
- Delivery ratio remains 1.0 across duplicate factors.
- Only extreme stress (very high duplication) shows drop below 1
- This drop is due to **BMv2 software buffer overflow**, not incorrect dedup logic
- **Takeaway**: FeBEx preserves unique uplinks reliably in realistic operating regimes

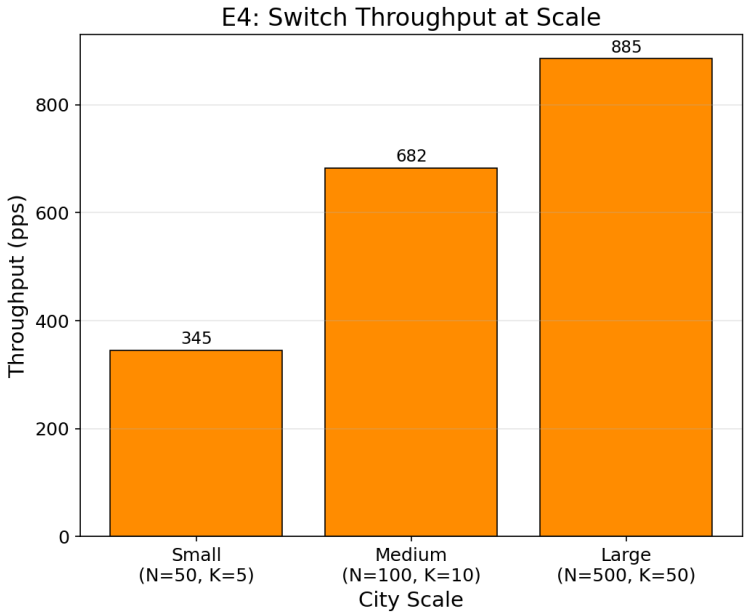


Params: N=100, K=10, M=2 | duplicate-factor sweep as in E1

6: City-scale Scalability

- Savings stay in a tight band (~62–67%) as topology scales from small to large
- Medium case can slightly exceed theoretical point estimates due to Poisson coverage randomness
- Throughput grows with scale and then plateaus at software-target limits
- **Takeaway**: dedup gain is scale-stable; bottleneck is implementation target, not FeBEx policy





Params: (N,K)=(50,5),(100,10),(200,20),(500,50) | M=4 | avg_cov≈3

7: Multi-tenant Isolation

- Cross-tenant leakage is zero in observed runs
- Isolation remains stable while dedup and forwarding are active together
- **Takeaway**: FeBEx enforces clean tenant separation

8: Receipt/Payment Accuracy

- Cloud receipts match reported gw_id values
- First-forwarder attribution works as intended for payment/accounting flow
- **Takeaway**: receipt mirroring is accurate and operationally usable

Verified from logs: results/**/lns*.tsv and cloud_receipts.tsv | Params: E3(N=100,K=10,M=4), E7(N=50,K=5,M=2), dedup ON

Thank you. Questions?